

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 6

Roll no: 38 PRN No: 12413458 Name of Student: **Vedanti Raut**

Subject Name : Deep Learning

Problem Statement : Design and implement a Convolutional Neural Network (CNN) for image classification using the Tomato or Soybean disease dataset.

Algorithm :

1. Import the required libraries for image processing, visualization, CNN implementation, and performance evaluation.
2. Download and load the New Plant Diseases Dataset containing multiple plant species and disease classes.
3. Identify the training and validation directories and load the images using `image_dataset_from_directory()`.
4. Resize all images to 64×64 pixels and divide them into batches.
5. Identify the different disease classes and visualize sample images from the dataset.
6. Analyze and visualize the distribution of images among the different disease classes.
7. Normalize the image pixel values from the range 0–255 to 0–1.
8. Construct a Convolutional Neural Network consisting of:
 - a. Convolutional layers for feature extraction
 - b. Max-pooling layers for dimensionality reduction
 - c. Global Average Pooling
 - d. Dense layer for classification
 - e. Softmax output layer for multiclass classification
9. Compile the CNN using the Adam optimizer, sparse categorical cross-entropy loss function, and accuracy as the evaluation metric.
10. Train the CNN using the training dataset for a specified number of epochs and validate it using the validation dataset.
11. Record the training and validation accuracy and loss during the training process.
12. Visualize the training and validation accuracy to analyze the learning performance of the model.
13. Visualize the training and validation loss to analyze model convergence and possible overfitting.
14. Evaluate the trained CNN using the validation dataset and calculate the final validation accuracy and loss.
15. Generate predictions for the validation images using the trained CNN.
16. Generate a classification report containing precision, recall, F1-score, and support for each disease class.
17. Generate and visualize the confusion matrix to analyze correctly and incorrectly classified plant disease classes.
18. Analyze the final accuracy and evaluation results to determine the overall performance of the CNN.

Code :

```
!kaggle datasets download -d vipooooo1/new-plant-diseases-dataset
```

```
Dataset URL: https://www.kaggle.com/datasets/vip000ool/new-plant-diseases-dataset  
License(s): copyright-authors  
Downloading new-plant-diseases-dataset.zip to /content  
100% 2.70G/2.70G [00:15<00:00, 192MB/s]
```

```
train_ds = tf.keras.utils.image_dataset_from_directory(
    train_dir,
    image_size=img_size,
    batch_size=batch_size,
    shuffle=True,
```

```

train_ds = tf.keras.utils.image_dataset_from_directory(
    train_dir,
    image_size=img_size,
    batch_size=batch_size,
    shuffle=True,
    seed=42
)

valid_ds = tf.keras.utils.image_dataset_from_directory(
    valid_dir,
    image_size=img_size,
    batch_size=batch_size,
    shuffle=False
)

class_names = train_ds.class_names

print("Number of Classes:", len(class_names))
print(class_names)

Found 70295 files belonging to 38 classes.
Found 17572 files belonging to 38 classes.
Number of Classes: 38
['Apple__Apple_scab', 'Apple__Black_rot', 'Apple__Cedar_apple_rust', 'Apple__healthy', 'Blueberry__healthy', 'Che

```

```

plt.figure(figsize=(10, 8))

for images, labels in train_ds.take(1):
    for i in range(9):
        plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")

```

```

import os
import matplotlib.pyplot as plt

class_counts = {}

for class_name in class_names:
    class_path = os.path.join(train_dir, class_name)

    count = len([
        f for f in os.listdir(class_path)
        if f.lower().endswith(('.jpg', '.jpeg', '.png'))
    ])

    class_counts[class_name] = count

plt.figure(figsize=(14, 6))

plt.bar(
    class_counts.keys(),
    class_counts.values()
)

plt.xlabel("Disease Class")
plt.ylabel("Number of Images")
plt.title("Class Distribution")

plt.xticks(rotation=90)
plt.tight_layout()
plt.show()

```

```
normalization_layer = tf.keras.layers.Rescaling(1./255)
```

```
train_ds = train_ds.map(  
    lambda x, y: (normalization_layer(x), y)  
)
```

```
valid_ds = valid_ds.map(  
    lambda x, y: (normalization_layer(x), y)  
)
```

```
AUTOTUNE = tf.data.AUTOTUNE
```

```
train_ds = train_ds.prefetch(AUTOTUNE)  
valid_ds = valid_ds.prefetch(AUTOTUNE)
```

```
model = Sequential([  
    Conv2D(16, (3, 3), activation='relu', input_shape=(64, 64, 3)),  
    MaxPooling2D((2, 2)),  
  
    Conv2D(32, (3, 3), activation='relu'),  
    MaxPooling2D((2, 2)),  
  
    Conv2D(64, (3, 3), activation='relu'),  
    MaxPooling2D((2, 2)),  
  
    GlobalAveragePooling2D(),  
  
    Dense(64, activation='relu'),  
  
    Dense(len(class_names), activation='softmax')  
)  
  
model.summary()
```

```
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

```
history = model.fit(  
    train_ds,  
    validation_data=valid_ds,  
    epochs=5  
)
```

```
plt.figure(figsize=(7, 5))
```

```
plt.plot(  
    history.history['accuracy'],  
    label='Training Accuracy'  
)
```

```
plt.plot(  
    history.history['val_accuracy'],  
    label='Validation Accuracy'  
)
```

```
plt.xlabel("Epoch")  
plt.ylabel("Accuracy")  
plt.title("Training vs Validation Accuracy")
```

```
plt.legend()  
plt.grid(True)
```

```
plt.show()
```

```
plt.figure(figsize=(7, 5))

plt.plot(
    history.history['loss'],
    label='Training Loss'
)

plt.plot(
    history.history['val_loss'],
    label='Validation Loss'
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss")

plt.legend()
plt.grid(True)

plt.show()
```

```
loss, accuracy = model.evaluate(valid_ds)

print("Validation Accuracy:", accuracy)
print("Validation Loss:", loss)

275/275 ————— 31s 112ms/step - accuracy: 0.7614 - loss: 0.7550
Validation Accuracy: 0.7614386677742004
Validation Loss: 0.755033016204834
```

```
y_true = []
y_pred = []

for images, labels in valid_ds:
    predictions = model.predict(images, verbose=0)

    y_true.extend(labels.numpy())
    y_pred.extend(np.argmax(predictions, axis=1))

y_true = np.array(y_true)
y_pred = np.array(y_pred)
```

```
print("Classification Report:\n")

print(
    classification_report(
        y_true,
        y_pred,
        target_names=class_names
    )
)
```

```
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(14, 12))

sns.heatmap(
    cm,
    cmap='Blues',
    xticklabels=class_names,
    yticklabels=class_names
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.xticks(rotation=90)
plt.yticks(rotation=0)

plt.tight_layout()
plt.show()
```

Output :

1. Sample Images

Tomato__Tomato_mosaic_virus



Corn_(maize)__Northern_Leaf_Blight



Tomato__Early_blight



Grape__Leaf_blight_(Isariopsis_Leaf_Spot)



Grape__Black_rot



Peach__Bacterial_spot



Blueberry__healthy



Pepper,_bell__healthy



Tomato__Septoria_leaf_spot



2. Training vs Validation A



3. Training vs Validation Loss



4. Classification Report

Classification Report:

	precision	recall	f1-score	support
Apple__Apple_scab	0.63	0.80	0.71	504
Apple__Black_rot	0.80	0.79	0.79	497
Apple__Cedar_apple_rust	0.69	0.60	0.64	440
Apple__healthy	0.82	0.61	0.70	502
Blueberry__healthy	0.75	0.87	0.81	454
Cherry_(including_sour)__Powdery_mildew	0.53	0.82	0.64	421
Cherry_(including_sour)__healthy	0.90	0.88	0.89	456
Corn_(maize)__Cercospora_leaf_spot Gray_leaf_spot	0.72	0.65	0.68	410
Corn_(maize)__Common_rust_	0.95	0.95	0.95	477
Corn_(maize)__Northern_Leaf_Blight	0.82	0.84	0.83	477
Corn_(maize)__healthy	0.92	0.99	0.96	465
Grape__Black_rot	0.62	0.77	0.69	472
Grape__Esca_(Black_Measles)	0.91	0.66	0.77	480
Grape__Leaf_blight_(Isariopsis_Leaf_Spot)	0.74	0.94	0.83	430
Grape__healthy	0.70	0.94	0.80	423
Orange__Haunglongbing_(Citrus_greening)	0.88	0.90	0.89	503
Peach__Bacterial_spot	0.92	0.50	0.65	459
Peach__healthy	0.87	0.90	0.88	432
Pepper,_bell__Bacterial_spot	0.62	0.64	0.63	478
Pepper,_bell__healthy	0.84	0.60	0.70	497
Potato__Early_blight	0.84	0.90	0.87	485
Potato__Late_blight	0.51	0.77	0.61	485
Potato__healthy	0.72	0.78	0.75	456
Raspberry__healthy	0.80	0.85	0.82	445
Soybean__healthy	0.91	0.64	0.75	505
Squash__Powdery_mildew	0.81	0.86	0.83	434
Strawberry__Leaf_scorch	0.85	0.77	0.81	444
Strawberry__healthy	0.84	0.95	0.89	456
Tomato__Bacterial_spot	0.65	0.77	0.71	425
Tomato__Early_blight	0.66	0.45	0.53	480
Tomato__Late_blight	0.66	0.43	0.52	463
Tomato__Leaf_Mold	0.70	0.68	0.69	470
Tomato__Septoria_leaf_spot	0.73	0.34	0.46	436
Tomato__Spider_mites Two-spotted_spider_mite	0.68	0.77	0.72	435
Tomato__Target_Spot	0.71	0.62	0.66	457
Tomato__Tomato_Yellow_Leaf_Curl_Virus	0.80	0.91	0.85	490
Tomato__Tomato_mosaic_virus	0.95	0.95	0.95	448
Tomato__healthy	0.89	0.86	0.87	481
accuracy			0.76	17572
macro avg	0.77	0.76	0.76	17572
weighted avg	0.77	0.76	0.76	17572

5. Confusion Matrix

Confusion Matrix

